



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ	Информатика и системы управления (ИУ)
КАФЕДРА	Информационная безопасность (ИУ8)
ДИСЦИПЛИНА	Математические основы искусственного интеллекта

ДОМАШНЕЕ ЗАДАНИЕ №4

Разработка многослойного персептрона
на основе обратного распространения ошибки FFNN

название

Группа ИУ8-13М

Студент	5 вариант		Никулина З.Е.
		<i>подпись</i>	<i>фамилия, и.о.</i>

Преподаватель		Каганов Ю.Т.
	<i>подпись</i>	<i>фамилия, и.о.</i>

Москва, 2025 г.

Цель работы – изучение многослойного персептрона, исследование его работы на основе использования различных методов оптимизации и целевых функций.

Практическая часть

1 Постановка задачи

Реализовать на языке высокого уровня многослойный персептрон и проверить его работоспособность на примере данных, выбранных из MNIST dataset. Исследовать работу персептрона на основе использования различных целевых функций. (среднеквадратичная ошибка, перекрестная энтропия, дивергенция Кульбака-Лейблера). Исследовать работу многослойного персептрона с использованием различных методов оптимизации (градиентный, Флетчера-Ривза (FR), Бройдена-Флетчера-Гольдфарба-Шенно (BFGS)). Провести исследование эффективности работы многослойного персептрона при изменении гиперпараметров (количества нейронов и количества слоев).

2 Реализация многослойного персептрона

На рисунке 1 показаны изображения рукописных цифр от 0 до 9 из датасета MNIST, содержащего 70 000 изображений размером 28×28 пикселей в оттенках серого. Для обучения используется 60000 изображений, для тестирования – 10000. В работе изображения нормализуются и преобразуются в векторы размерности 784 признака для последующего использования в многослойном персептроне. На рисунке 2 показана случайная выборка этих цифр.

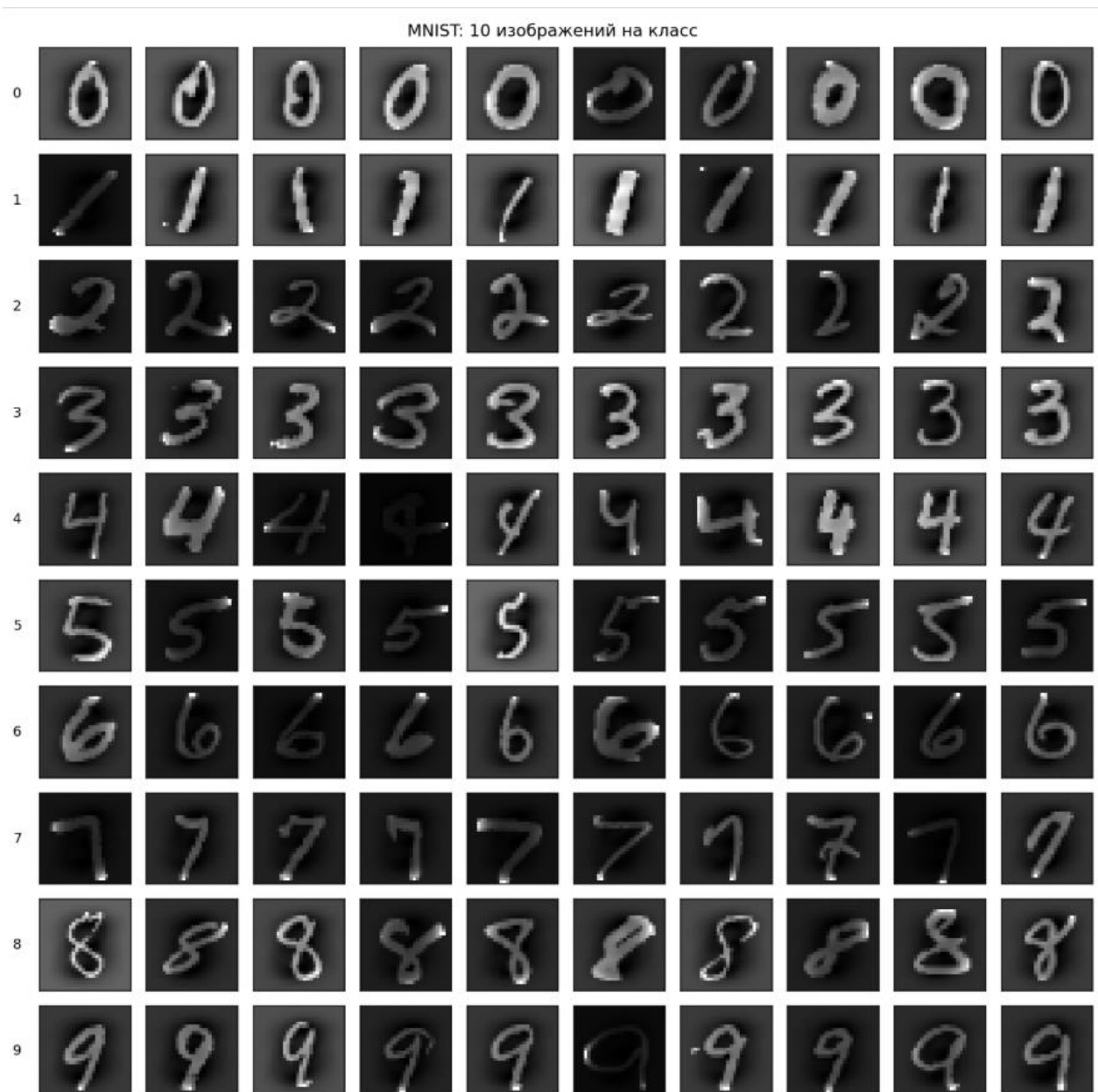


Рисунок 1 – Изображения из MNIST



Рисунок 2 – Случайная выборка

На рисунке 3-4 представлены зависимости функции потерь от числа эпох для трёх различных целевых функций при обучении многослойного персептрона с использованием градиентного спуска и функции активации ReLU. Все варианты демонстрируют убывание значения потерь с ростом числа эпох, что указывает на успешное обучение модели. Кросс-энтропия и дивергенция Кульбака–Лейблера имеют схожую динамику и снижаются быстрее в начальных итерациях, в то время как среднеквадратичная ошибка убывает более плавно, но достигает меньших абсолютных значений, что связано с особенностями этой функции потерь при решении задач классификации.

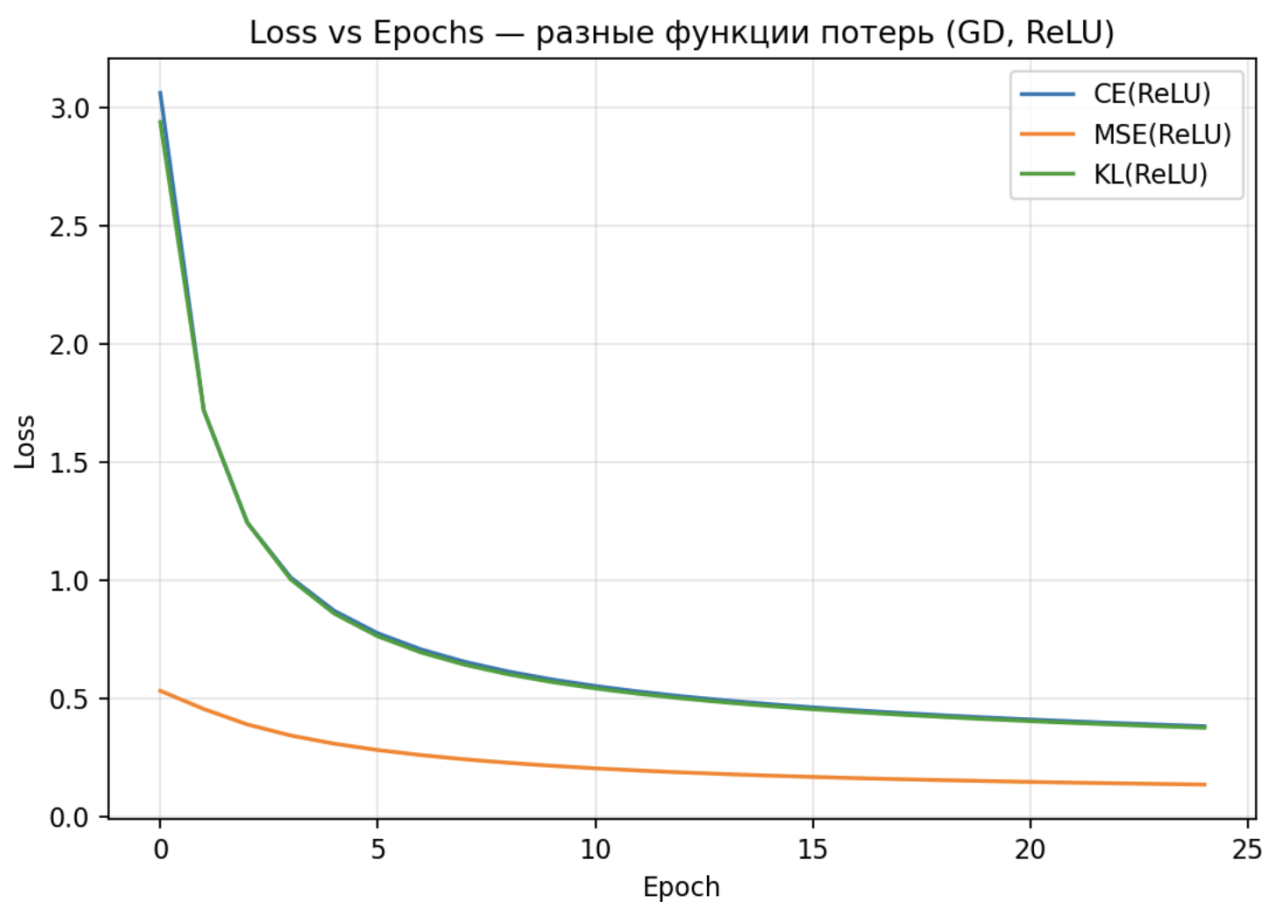


Рисунок 3 – График зависимости функции потерь от числа эпох

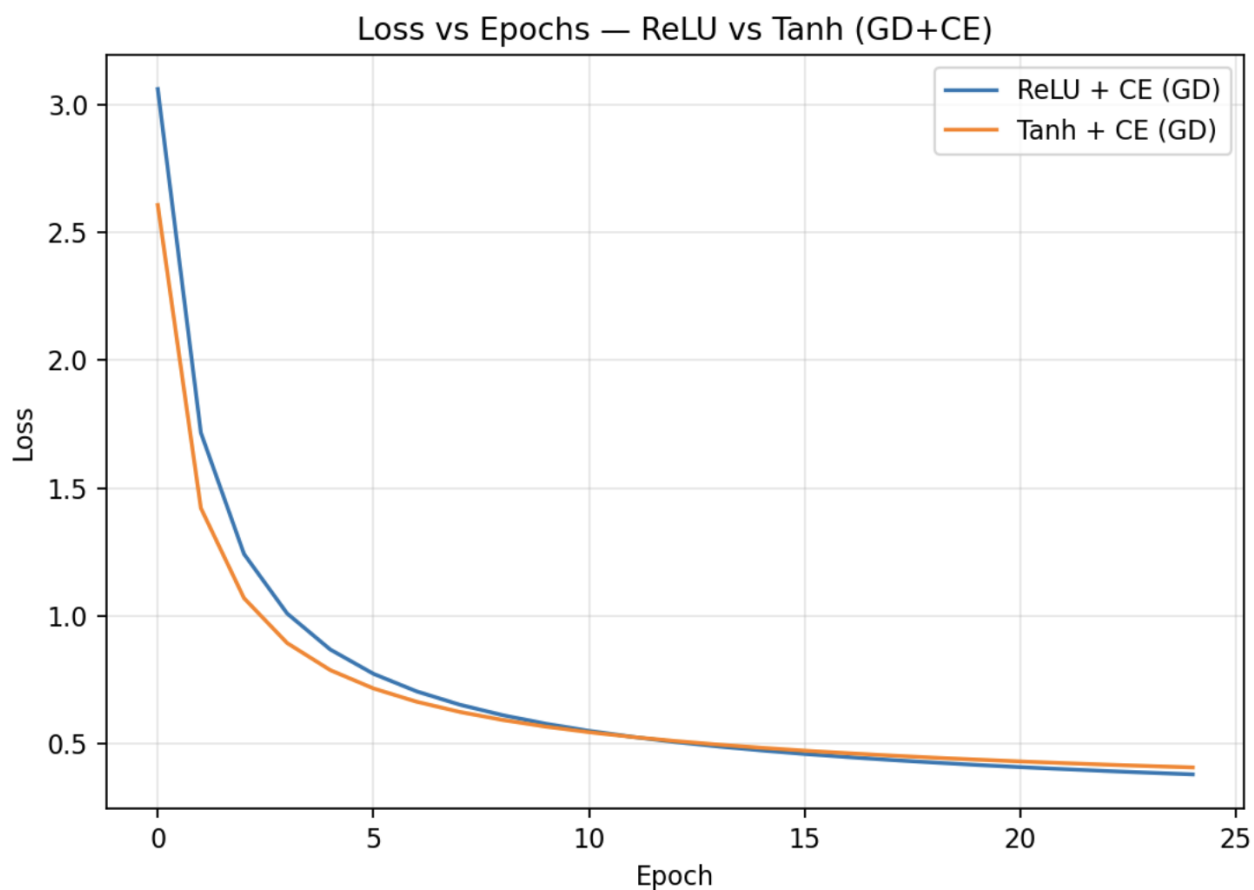


Рисунок 4 – График динамики обучения при различных ФА

Матрица результатов классификации на тестовой выборке для лучшей модели (BFGS + CE + ReLU) показана на рисунке 5. Большинство элементов сосредоточено на диагонали, что указывает на высокую точность распознавания всех десяти классов. Нормированная матрица (рисунок 6) показывает долю правильных предсказаний по каждому классу: значения варьируются от 0.91 до 0.98, что свидетельствует о качественном разделении символов. Наибольшее число ошибок наблюдается для цифр 4, 5, 8 и 9, что типично для рукописных данных из-за сходства их формы. В целом модель демонстрирует высокую обобщающую способность и устойчивую классификацию большинства записанных вручную цифр.

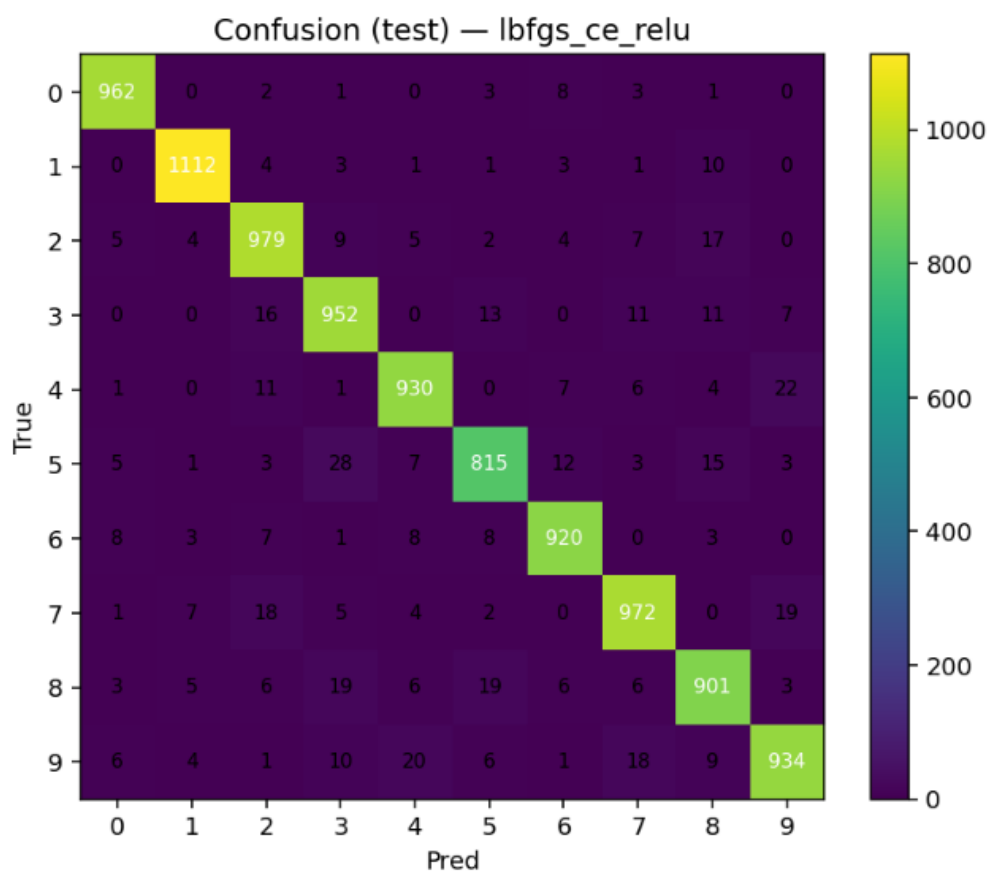


Рисунок 5 – Классификация для лучшей модели

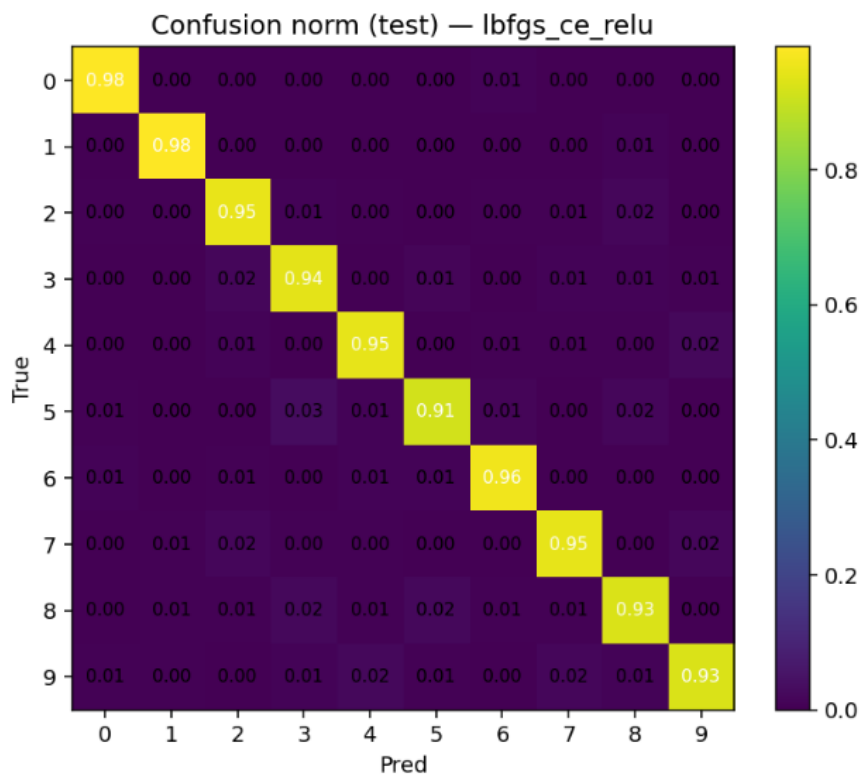


Рисунок 6 – Нормированная матрица

Примеры предсказаний (на рисунке 7) демонстрируют работу модели на тестовой выборке: большинство изображений классифицировано корректно с высокой уверенностью, что подтверждает способность модели успешно распознавать рукописные цифры. Во втором наборе изображений на рисунке 8 приведены ошибки классификации – они, как правило, связаны с неоднозначными или плохо различимыми цифрами, имеющими нестандартное начертание, сильный наклон или шум. В таких случаях некоторые цифры похожи на предсказанные моделью классы (например, 9 - 4 или 5 - 3).



Рисунок 7 – Примеры предсказаний

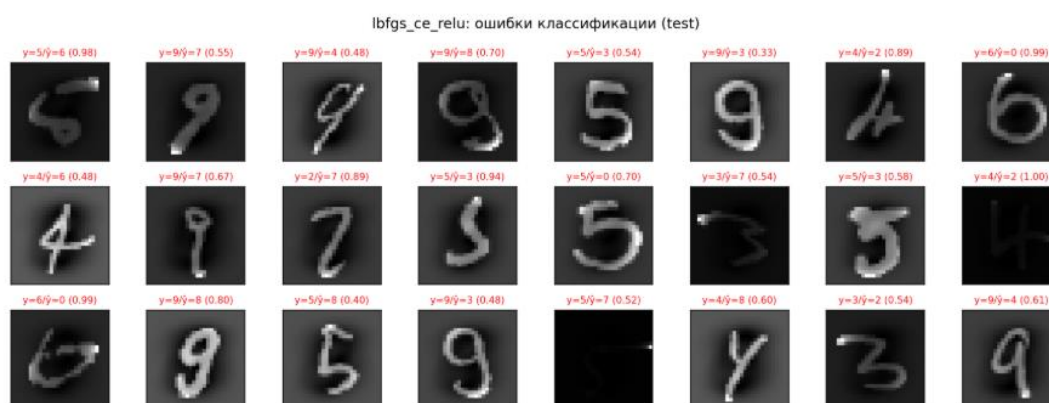


Рисунок 8 – Ошибки предсказаний

Текстовый отчёт на рисунке 9 по качеству классификации демонстрирует высокий уровень распознавания рукописных цифр моделью: точность (precision), полнота (recall) и F1-мера для большинства классов превышают 0.93, что подтверждает стабильность и надёжность работы классификатора. Наилучшие результаты наблюдаются для классов цифр 1, 6 и 7, где показатели

уверенно держатся около 0.97–0.98. Несколько ниже значения метрик для цифр 5, 8 и особенно 9 — эти классы чаще путаются с визуально похожими цифрами. Итоговая точность по тестовой выборке составляет около 0.937, а макро- и взвешенное среднее F1 также находятся на уровне 0.94, что указывает на сбалансированное качество классификации по всем классам и успешную генерализацию модели на невидимые данные.

Отчет (test):				
	precision	recall	f1-score	support
0	0.9615	0.9694	0.9654	980
1	0.9780	0.9780	0.9780	1135
2	0.9359	0.9196	0.9277	1032
3	0.9341	0.9257	0.9299	1010
4	0.9450	0.9277	0.9363	982
5	0.9135	0.9114	0.9125	892
6	0.9360	0.9614	0.9485	958
7	0.9448	0.9319	0.9383	1028
8	0.8875	0.9230	0.9049	974
9	0.9258	0.9148	0.9202	1009
accuracy			0.9369	10000
macro avg	0.9362	0.9363	0.9362	10000
weighted avg	0.9371	0.9369	0.9369	10000

Рисунок 9 – Метрики предсказаний

На рисунке 10 проведено исследование влияния архитектуры сети на качество классификации показало, что увеличение ширины слоя (количества нейронов) оказывает положительное влияние на точность. Наилучший результат получен для модели с одной скрытой прослойкой из 256 нейронов, которая обеспечила максимальную валидационную точность и минимальные значения функции потерь. Был получен вектор синаптических весов (рисунок 11) лучшей модели длиной 101 770 параметров, что подтверждает высокую модельную сложность даже при относительно компактной архитектуре.

	layers	width	val_acc	loss
0	1	64	0.864667	0.514439
1	1	128	0.865333	0.526730
2	1	256	0.868667	0.509353
3	2	64	0.848083	0.574748
4	2	128	0.863583	0.522232
5	2	256	0.864083	0.527707
6	3	64	0.840417	0.601052
7	3	128	0.775417	0.634009
8	3	256	0.842167	0.614062

Рисунок 10 – Влияние архитектуры на точность

```
Вектор синаптических весов лучшей модели: длина = 101770
Первые 120 элементов: [ 0.07334386  0.04635581  0.1289367 ... -0.06130201 -0.02487472  0.05852756]
...
Последние 120 элементов: [-0.02044274 -0.01749035 -0.02803828 ... -0.08029827  0.03099634  0.08392162]
```

Рисунок 11 – Вектор найденных синаптических весов

Листинги программ приведены в Приложении А.

ПРИЛОЖЕНИЕ А

Листинги программ

```
import os
import urllib.request
from dataclasses import dataclass
from typing import List, Dict, Tuple

import numpy as np
from numpy.random import default_rng

def load_mnist() -> Tuple[np.ndarray, np.ndarray, np.ndarray, np.ndarray]:
    try:
        from azureml.opendatasets import MNIST # noqa: F401
        tr = MNIST.get_tabular_dataset(dataset_filter="train").to_pandas_dataframe()
        te = MNIST.get_tabular_dataset(dataset_filter="test").to_pandas_dataframe()
        Xtr = tr.drop("label", axis=1).astype("float32").to_numpy() / 255.0
        ytr = tr["label"].astype(int).to_numpy()
        Xte = te.drop("label", axis=1).astype("float32").to_numpy() / 255.0
        yte = te["label"].astype(int).to_numpy()
        print("[INFO] MNIST загружен через Azure Open Datasets.")
        return Xtr, ytr, Xte, yte
    except Exception as e:
        print(f"[WARN] Azure Open Datasets недоступен: {repr(e)}")
    try:
        os.makedirs("data", exist_ok=True)
        npz_path = os.path.join("data", "mnist.npz")
        if not os.path.exists(npz_path):
            url = "https://storage.googleapis.com/tensorflow/tf-keras-
datasets/mnist.npz"
            print("[INFO] Скачиваю mnist.npz ...")
            urllib.request.urlretrieve(url, npz_path)
        with np.load(npz_path) as f:
            Xtr, ytr = f["x_train"], f["y_train"]
            Xte, yte = f["x_test"], f["y_test"]
            Xtr = Xtr.reshape(-1, 28 * 28).astype("float32") / 255.0
            Xte = Xte.reshape(-1, 28 * 28).astype("float32") / 255.0
            print("[INFO] MNIST загружен из mnist.npz.")
            return Xtr, ytr.astype(int), Xte, yte.astype(int)
    except Exception as e:
        print(f"[WARN] mnist.npz недоступен: {repr(e)}")
    try:
        from sklearn.datasets import fetch_openml # noqa: F401
        ds = fetch_openml(data_id=554, as_frame=False) # mnist_784
        X = ds["data"].astype("float32") / 255.0
        y = ds["target"].astype(int)
        Xtr, ytr = X[:60000], y[:60000]
        Xte, yte = X[60000:], y[60000:]
        print("[INFO] MNIST загружен через OpenML (data_id=554).")
        return Xtr, ytr, Xte, yte
    except Exception as e:
        raise RuntimeError("Не удалось загрузить MNIST ни одним способом.") from e

RNG = default_rng(42)

def one_hot(y: np.ndarray, n: int = 10) -> np.ndarray:
    Z = np.zeros((y.size, n), dtype=np.float32)
    Z[np.arange(y.size), y] = 1.0
```

```

    return Z

def standardize(X: np.ndarray):
    mu = X.mean(axis=0, keepdims=True)
    sigma = X.std(axis=0, keepdims=True) + 1e-8
    return (X - mu) / sigma, mu, sigma

def train_val_split(X, y, val_ratio=0.2):
    idx = np.arange(X.shape[0]); RNG.shuffle(idx)
    cut = int(X.shape[0] * (1 - val_ratio))
    return X[idx[:cut]], y[idx[:cut]], X[idx[cut:]], y[idx[cut:]]

@dataclass
class MLPConfig:
    input_dim: int = 784
    hidden_layers: List[int] = None
    output_dim: int = 10
    activation: str = "relu"
    l2: float = 1e-4

class MLP:
    def __init__(self, cfg: MLPConfig):
        self.cfg = cfg
        hs = cfg.hidden_layers or [128]
        dims = [cfg.input_dim] + hs + [cfg.output_dim]

        self.W: List[np.ndarray] = []
        self.b: List[np.ndarray] = []

        for i in range(len(dims) - 1):
            fin, fout = dims[i], dims[i + 1]
            if i < len(hs): # скрытые слои
                scale = np.sqrt(2.0 / fin) if cfg.activation == "relu" else 1.0 /
np.sqrt(fin)
            else: # выходной
                scale = np.sqrt(2.0 / fin)
            self.W.append(RNG.normal(0, scale, size=(fin, fout)).astype(np.float32))
            self.b.append(np.zeros((1, fout), dtype=np.float32))

        # упаковка/распаковка параметров (для CG/L-BFGS)
        def params_vector(self) -> np.ndarray:
            return np.concatenate([w.ravel() for w in self.W] + [b.ravel() for b in
self.b])

        def set_params_vector(self, v: np.ndarray):
            idx = 0
            for i in range(len(self.W)):
                fin, fout = self.W[i].shape
                self.W[i] = v[idx:idx + fin * fout].reshape(fin, fout).astype(np.float32)
                idx += fin * fout
            for i in range(len(self.b)):
                fout = self.b[i].shape[1]
                self.b[i] = v[idx:idx + fout].reshape(1, fout).astype(np.float32)
                idx += fout

        def forward(self, X: np.ndarray) -> Dict[str, List[np.ndarray]]:
            h = X
            H, masks = [h], []
            # скрытые
            for i in range(len(self.W) - 1):
                z = h @ self.W[i] + self.b[i]

```

```

        if self.cfg.activation == "relu":
            h = np.maximum(0, z); mask = (z > 0).astype(np.float32)
        else:
            h = np.tanh(z); mask = 1.0 - h ** 2
        H.append(h); masks.append(mask)
        # выходной softmax (стабильный)
        zL = h @ self.W[-1] + self.b[-1]
        zLm = zL - np.max(zL, axis=1, keepdims=True)
        expz = np.exp(np.clip(zLm, -50, 50))
        yhat = expz / (np.sum(expz, axis=1, keepdims=True) + 1e-12)
        return {"H": H, "masks": masks, "yhat": yhat}

def loss_and_grad(model: MLP, X: np.ndarray, Y: np.ndarray, loss_name="cross_entropy"):
    out = model.forward(X)
    H, masks, yhat = out["H"], out["masks"], out["yhat"]

    if loss_name == "cross_entropy":
        eps = 1e-12
        loss = -np.sum(Y * np.log(yhat + eps)) / X.shape[0]
        dZ = (yhat - Y) / X.shape[0]
    elif loss_name == "mse":
        diff = yhat - Y
        loss = 0.5 * np.sum(diff ** 2) / X.shape[0]
        dZ = diff / X.shape[0]
    elif loss_name == "kl":
        eps = 1e-12
        loss = np.sum(Y * (np.log(Y + eps) - np.log(yhat + eps))) / X.shape[0]
        dZ = (yhat - Y) / X.shape[0]
    else:
        raise ValueError("Unknown loss")

    # L2
    if model.cfg.l2 > 0:
        loss += 0.5 * model.cfg.l2 * sum(np.sum(w * w) for w in model.W)

    # градиенты
    dW = [None] * len(model.W)
    db = [None] * len(model.b)

    dW[-1] = H[-1].T @ dZ + model.cfg.l2 * model.W[-1]
    db[-1] = dZ.sum(axis=0, keepdims=True)
    dh = dZ @ model.W[-1].T

    for i in range(len(model.W) - 2, -1, -1):
        dh = dh * masks[i]
        dW[i] = H[i].T @ dh + model.cfg.l2 * model.W[i]
        db[i] = dh.sum(axis=0, keepdims=True)
        if i > 0:
            dh = dh @ model.W[i].T

    g = np.concatenate([dW[i].ravel() for i in range(len(dW))] + [db[i].ravel() for i
in range(len(db))])
    return float(loss), g

def accuracy(model: MLP, X: np.ndarray, y: np.ndarray) -> float:
    return float((np.argmax(model.forward(X)["yhat"], axis=1) == y).mean())

def armijo(model, X, Y, loss_name, d, g, c=1e-4, tau=0.5, it=20):
    w0 = model.params_vector()
    f0, _ = loss_and_grad(model, X, Y, loss_name)
    if np.dot(g, d) >= 0:

```

```

        d = -g
    t = 1.0
    for _ in range(it):
        model.set_params_vector(w0 + t * d)
        f1, _ = loss_and_grad(model, X, Y, loss_name)
        if f1 <= f0 + c * t * np.dot(g, d):
            return t
        t *= tau
    model.set_params_vector(w0)
    return t

def train_gd(model, X, Y, Xv, yv, loss_name, lr=0.05, epochs=25, clip_norm=5.0):
    #Градиентный
    hist = {"loss": [], "val_acc": []}
    for ep in range(epochs):
        L, g = loss_and_grad(model, X, Y, loss_name)
        g_norm = np.linalg.norm(g)
        if g_norm > clip_norm:
            g = g * (clip_norm / (g_norm + 1e-12))
        model.set_params_vector(model.params_vector() - lr * g)
        hist["loss"].append(float(L))
        hist["val_acc"].append(accuracy(model, Xv, yv))
        print(f"[GD] epoch={ep+1:02d} loss={hist['loss'][-1]:.4f} val_acc={hist['val_acc'][-1]:.4f}")
    return hist

def train_cg_fr(model, X, Y, Xv, yv, loss_name, epochs=20):
    #Сопряжённые градиенты (Флетчер-Ривз)
    hist = {"loss": [], "val_acc": []}
    L, g_prev = loss_and_grad(model, X, Y, loss_name)
    d = -g_prev
    for ep in range(epochs):
        step = armijo(model, X, Y, loss_name, d, g_prev)
        w = model.params_vector()
        model.set_params_vector(w + step * d)
        L, g = loss_and_grad(model, X, Y, loss_name)
        beta = np.dot(g, g) / (np.dot(g_prev, g_prev) + 1e-12)
        d = -g + beta * d
        g_prev = g
        hist["loss"].append(L)
        hist["val_acc"].append(accuracy(model, Xv, yv))
        print(f"[CG-FR] epoch={ep+1:02d} loss={L:.4f} val_acc={hist['val_acc'][-1]:.4f}")
    return hist

def _lbfgs_two_loop(S: List[np.ndarray], Yv: List[np.ndarray], g: np.ndarray) -> np.ndarray:
    q = g.copy()
    m = len(S)
    alpha, rho = [], []
    for i in range(m - 1, -1, -1):
        s, y = S[i], Yv[i]
        rho_i = 1.0 / (np.dot(y, s) + 1e-12)
        rho.append(rho_i)
        a = rho_i * np.dot(s, q)
        alpha.append(a)
        q = q - a * y
    gamma = 1.0
    if m > 0:
        yL, sL = Yv[-1], S[-1]
        gamma = np.dot(sL, yL) / (np.dot(yL, yL) + 1e-12)
    r = gamma * q

```

```

    for i in range(m):
        s, y = S[i], Yv[i]
        b = rho[m - 1 - i] * np.dot(y, r)
        r = r + s * (alpha[m - 1 - i] - b)
    return -r

def train_lbfgs(model, X, Y, Xv, yv, loss_name, epochs=15, m_hist=10):
    #BFGS
    hist = {"loss": [], "val_acc": []}
    S, Yv = [], []
    L, g_prev = loss_and_grad(model, X, Y, loss_name)
    for ep in range(epochs):
        d = _lbfgs_two_loop(S, Yv, g_prev)
        step = armijo(model, X, Y, loss_name, d, g_prev)
        w = model.params_vector()
        model.set_params_vector(w + step * d)
        L2, g2 = loss_and_grad(model, X, Y, loss_name)
        s = model.params_vector() - w
        y = g2 - g_prev
        S.append(s); Yv.append(y)
        if len(S) > m_hist:
            S.pop(0); Yv.pop(0)
        g_prev = g2
        hist["loss"].append(L2)
        hist["val_acc"].append(accuracy(model, Xv, yv))
        print(f"[L-BFGS] epoch={ep+1:02d} loss={L2:.4f} val_acc={hist['val_acc'][-1]:.4f}")
    return hist

def run_all():
    # загрузка
    Xtr, ytr, Xte, yte = load_mnist()

    # предобработка
    Xtr, mu, sigma = standardize(Xtr)
    Xte = (Xte - mu) / sigma
    Xtr, ytr, Xval, yval = train_val_split(Xtr, ytr, val_ratio=0.2)
    Ytr, Yval = one_hot(ytr, 10), one_hot(yval, 10)

    print("\nЭкспериментный стенд")
    print("Train:", Xtr.shape, ytr.shape, "| Val:", Xval.shape, yval.shape, "| Test:",
Xte.shape, yte.shape)

    # Базовые конфигурации
    cfg_relu = MLPConfig(hidden_layers=[128], activation="relu", l2=1e-4)
    cfg_tanh = MLPConfig(hidden_layers=[128], activation="tanh", l2=1e-4)

    # (2) Функции потерь (GD)
    print("\nСравнение функций потерь: GD + ReLU")
    m_ce = MLP(cfg_relu); h_ce = train_gd(m_ce, Xtr, Ytr, Xval, yval,
loss_name="cross_entropy", lr=0.05, epochs=20)
    m_mse = MLP(cfg_relu); h_mse = train_gd(m_mse, Xtr, Ytr, Xval, yval,
loss_name="mse", lr=0.05, epochs=20)
    m_kl = MLP(cfg_relu); h_kl = train_gd(m_kl, Xtr, Ytr, Xval, yval, loss_name="kl",
lr=0.05, epochs=20)

    # (3) Сравнение оптимизаторов (CE + ReLU)
    print("\nСравнение оптимизаторов: CE + ReLU")
    m_gd = MLP(cfg_relu); h_gd = train_gd(m_gd, Xtr, Ytr, Xval, yval,
"cross_entropy", lr=0.05, epochs=20)
    m_cg = MLP(cfg_relu); h_cg = train_cg_fr(m_cg, Xtr, Ytr, Xval, yval,
"cross_entropy", epochs=15)

```

```

m_lb = MLP(cfg_relu); h_lb = train_lbfgs(m_lb, Xtr, Ytr, Xval, yval,
"cross_entropy", epochs=12)

# ReLU vs Tanh (CE + GD)
print("\nReLU vs Tanh: GD + CE")
m_relu = MLP(cfg_relu); h_relu = train_gd(m_relu, Xtr, Ytr, Xval, yval,
"cross_entropy", lr=0.05, epochs=20)
m_tanh = MLP(cfg_tanh); h_tanh = train_gd(m_tanh, Xtr, Ytr, Xval, yval,
"cross_entropy", lr=0.05, epochs=20)

# Итоговые валидационные точности
def last(h): return h["val_acc"][-1]
print("\nИтог")
rows = [
    ("GD (CE, ReLU)", last(h_ce)),
    ("GD (MSE, ReLU)", last(h_mse)),
    ("GD (KL, ReLU)", last(h_kl)),
    ("GD (CE, ReLU)", last(h_gd)),
    ("CG-FR (CE, ReLU)", last(h_cg)),
    ("L-BFGS (CE, ReLU)", last(h_lb)),
    ("GD (CE, Tanh)", last(h_tanh)),
]
for name, va in rows:
    print(f"{name:22s} val_acc={va:.4f}")

best_model = max([(m_gd, last(h_gd), "GD"),
                  (m_cg, last(h_cg), "CG-FR"),
                  (m_lb, last(h_lb), "L-BFGS")], key=lambda x: x[1])[0]
best_tag = "best_ce_relu"
test_acc = accuracy(best_model, Xte, yte)
print(f"\n Лучшая модель (CE+ReLU, по валидации): test_acc={test_acc:.4f}")

from sklearn.metrics import confusion_matrix, classification_report
probs = best_model.forward(Xte)["yhat"]
y_pred = np.argmax(probs, axis=1)
cm = confusion_matrix(yte, y_pred, labels=list(range(10)))
print("\nConfusion matrix (test):")
print(cm)
print("\nОтчет (test):")
print(classification_report(yte, y_pred, digits=4))

# (4) Гиперпараметры: число слоёв x число нейронов (быстрый прогон)
print("\nВлияние гиперпараметров (GD+CE, ReLU)")
widths = [32, 64]
layers = [1, 2]
grid_results = []
for L in layers:
    for W in widths:
        cfg = MLPConfig(hidden_layers=[W] * L, activation="relu", l2=1e-4)
        m = MLP(cfg)
        h = train_gd(m, Xtr, Ytr, Xval, yval, "cross_entropy", lr=0.05, epochs=10)
        va = h["val_acc"][-1]
        grid_results.append((L, W, va))
        print(f"[HP] layers={L} width={W} -> val_acc={va:.4f}")
print("\nСводка по гиперпараметрам:")
for L, W, va in grid_results:
    print(f"layers={L} width={W} val_acc={va:.4f}")

w = best_model.params_vector()
PRINT_FULL_WEIGHTS = False # чтобы не заливать консоль сотнями тысяч чисел
print(f"\nВектор синаптических весов лучшей модели: длина = {w.size}")
if PRINT_FULL_WEIGHTS:

```

```
    np.set_printoptions(suppress=True, linewidth=140)
    print(w)
else:
    np.set_printoptions(suppress=True, linewidth=140, threshold=100)
    print("Первые 120 элементов:", w[:120])
    print("...")
    print("Последние 120 элементов:", w[-120:])
```